Pointers

addresses, `new` and `delete`, and what `this` really is

Last time: `this->hours`

To fix `setTime()`, you wrote lines 62 to 64. They worked: the left side of each line meant the instance variable.

But what *is* `this`? And what does `->` actually do?

Today: `this` is a pointer. By the end of class you'll be able to read line 62 completely.

```
54  bool Clock::setTime(int hours, int minutes,
55          int seconds) {
56      if (hours < 0 || hours > 23 ||
57              minutes < 0 || minutes > 59 ||
58              seconds < 0 || seconds > 59) {
59          return false;
60      }
61
62      this->hours = hours;
63      this->minutes = minutes;
64      this->seconds = seconds;
65      return true;
66  }
```

Review: pass by reference

The `&` in `int &first` makes `first` a **reference**: another name for whatever was passed in.

`addTen(a, b)` doesn't copy `a`. Inside `addTen()`, `first` *is* `a`.

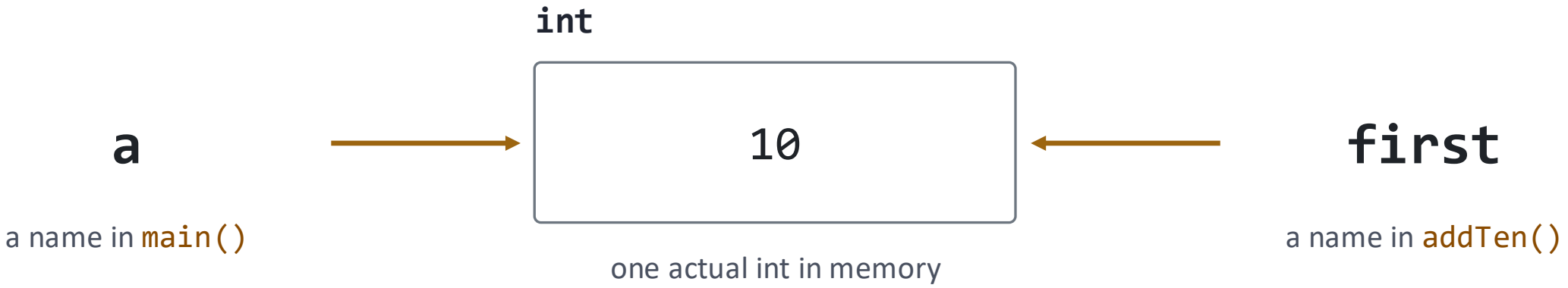
```
$ ./reference
0 1
10 11
```

```
1   #include <iostream>
2   using namespace std;
3
4   void addTen(int &first, int &second) {
5       first = first + 10;
6       second = second + 10;
7   }
8
9   int main() {
10      int a = 0;
11      int b = 1;
12
13      cout << a << " " << b << endl;
14      addTen(a, b);
15      cout << a << " " << b << endl;
16
17      return 0;
18  }
```

How does `addTen()` reach `a`?

A reference is another name for an existing variable. There's only one `int` here, with two names.

To make that work, the compiler usually passes `addTen()` the **address** of `a`: where `a` lives in memory. Today we work with addresses directly.



Change `first`, and you've changed `a`, because they're the same `int`.

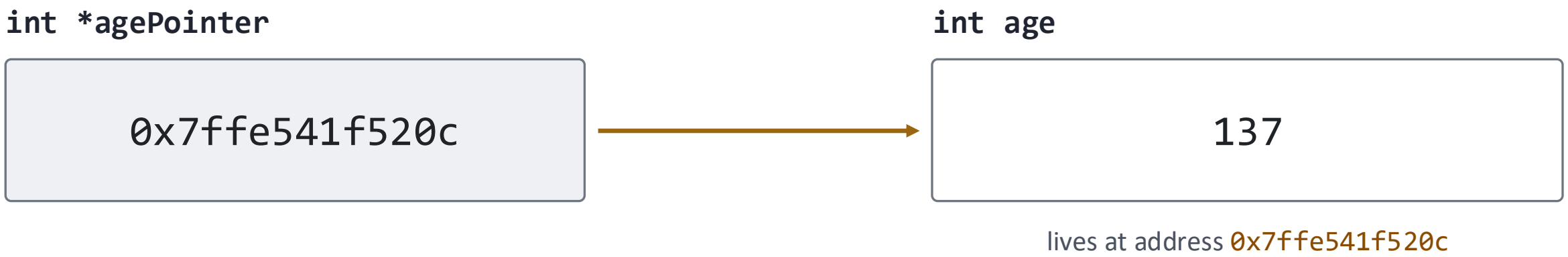
Addresses and pointers

every variable lives somewhere

What is a pointer?

Every variable lives somewhere in memory, and every spot in memory has an **address**: a number, usually written in hexadecimal.

A **pointer** is a variable that holds an address. It *points to* another variable.



`agePointer` doesn't hold 137. It holds *where to find* 137.

Pointers in code

`int *agePointer` declares a pointer to an `int`.

`&age` means *the address of* `age`.

`*agePointer` means *the thing it points to*. Change that (line 13), and you change `age`.

```
$ ./pointer
age: 137
&age: 0x7ffe541f520c
agePointer: 0x7ffe541f520c
*agePointer: 137
age: 42
```

```
4   int main() {
5       int age = 137;
6       int *agePointer = &age;
7
8       cout << "age: " << age << endl;
9       cout << "&age: " << &age << endl;
10      cout << "agePointer: " << agePointer << endl;
11      cout << "*agePointer: " << *agePointer << endl;
12
13      *agePointer = 42;
14      cout << "age: " << age << endl;
15
16      return 0;
17  }
```

Relevant xkcd: pointers



& and * each have two jobs

	In a declaration: part of the type	In an expression: an operator
`&`	`int &first = a;` `first` is a *reference* to an `int`	`&age` the *address of* `age`
`*`	`int *agePointer;` `agePointer` is a *pointer* to an `int`	`*agePointer` the thing `agePointer` *points to*

Line 6 of the last program uses both:

```
int *agePointer = &age;
```

`agePointer` is a pointer to an `int`, and it starts out holding the address of `age`.

Pointers have types

A pointer to an `int` holds the address of an `int`. A pointer to a `string` holds the address of a `string`.

Mixing them up (line 11) is a compile error. That's good news: the compiler catches it for you.

```
5   int main() {
6       int age = 137;
7       string name = "Ada";
8
9       int *agePointer = &age;
10      string *namePointer = &name;
11      int *oops = &name;
12
13      return 0;
```

```
$ g++ types.cpp
types.cpp:11:17: error: cannot convert 'std::string*' {aka 'std::__cxx11::basic_string<char>*'}
to 'int*' in initialization
   11 |     int *oops = &name;
      |                 ^~~~~
```

Predict: `int* a, b;`

Spacing doesn't matter to the compiler: `int* a` and `int *a` mean the same thing.

So on line 3, what are the types of `a` and `b`?

```
1   int main() {
2       int x = 5;
3       int* a, b;
4
5       a = &x;
6       b = &x;
7
8       return 0;
9   }
```

note: we haven't gone over declaring multiple variables on one line so far in class, 1) because it causes confusion when we get to pointers, 2) because most style guides discourage it *specifically because* it causes confusion with pointer declaration

Answer: only `a` is a pointer

The `*` belongs to `a`, not to `int`. `b` is a plain `int`, so line 6 doesn't compile.

Safest habit: **one declaration per line**.

This course writes `int *a`. Lots of code writes `int* a`. Both are fine on their own line.

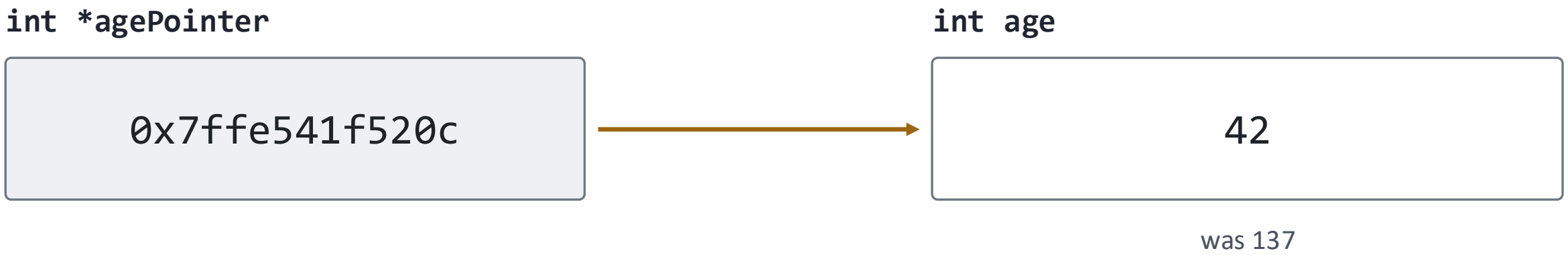
```
1   int main() {
2       int x = 5;
3       int* a, b;
4
5       a = &x;
6       b = &x;
7
8       return 0;
9   }
```

```
$ g++ twoPointers.cpp
twoPointers.cpp:6:9: error: invalid conversion from 'int*' to 'int' [-fpermissive]
    6 |     b = &x;
      |         ^~
```

Dereferencing: follow the arrow

`*agePointer` means *the thing it points to*. You can read it, or assign to it:

```
*agePointer = 42;
```



agePointer	means	the address it holds: unchanged by line 13
***agePointer**	means	age itself, which is now 42

When pointers go wrong

undefined behavior, and how to catch it

Predict: an uninitialized pointer

`brokenPointer` was never given an address. It holds whatever bits happened to be in that memory already.

What does line 10 print?

```
4   int main() {
5       int age = 137;
6       int *agePointer = &age;
7       int *brokenPointer;
8
9       cout << *agePointer << endl;
10      cout << *brokenPointer << endl;
11
12      return 0;
13  }
```

Answer: it depends

This is **undefined behavior**: the C++ standard doesn't say what happens, so *anything* can.

A crash is the lucky outcome, because at least you know. Printing a believable number and carrying on is worse.

g++ (Linux)

```
$ g++ broken.cpp
$ ./a.out
137
Segmentation fault
```

clang++ (Linux)

```
$ clang++ broken.cpp
$ ./a.out
137
-637171384
```

It's possible to get *basically any result* when you're dealing with undefined behavior

`nullptr`: pointing at nothing, on purpose

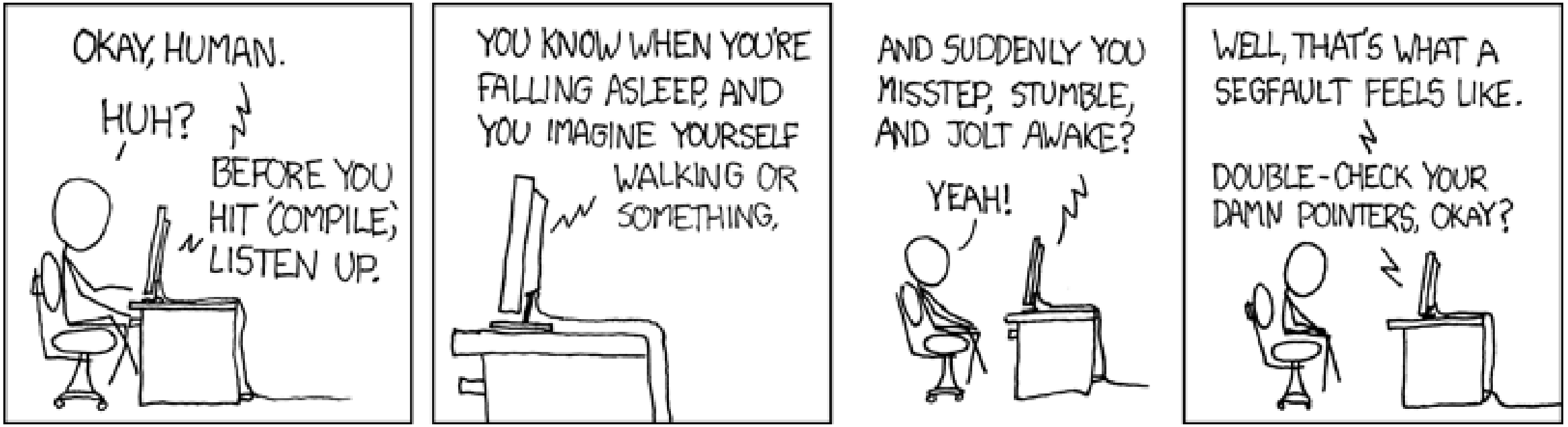
There's no way to test whether a pointer was initialized. So **always initialize pointers**: to a real address, or to `nullptr` if you don't have one yet.

You *can* test for `nullptr` (line 8), and you should before following a pointer that might be null.

```
5   int main() {
6       string *namePointer = nullptr;
7
8       if (namePointer == nullptr) {
9           cout << "That's a null pointer." << endl;
10          cout << "Not dereferencing it!" << endl;
11      } else {
12          cout << *namePointer << endl;
13      }
14
15      return 0;
16  }
```

```
$ ./nullptr
That's a null pointer.
Not dereferencing it!
```

Relevant xkcd: compiler complaint



References vs. pointers

	Reference: `int &r = a;`	**Pointer: `int *p = &a;`**
Can it be null?	No	Yes: `nullptr`
Must it be initialized?	Yes	No, but always do it anyway
Can it switch to another variable?	No	Yes: `p = &b;`
How do you reach `a`?	Just use `r`	`*p` (or `p->` for objects)

References are simpler and safer, so prefer them when they do the job. Pointers can do things references can't: point at nothing, point somewhere new, and reach objects made with `new` (coming up).

Pointers to objects

what `this` really is

`->`: follow a pointer to a member

`(*clockPointer).print24()` follows the pointer to the `Clock`, then calls `print24()` on it.

`clockPointer->print24()` means exactly the same thing, with less punctuation.

The parentheses matter: `*clockPointer.print24()` doesn't compile.

```
49  int main() {
50      Clock meeting(9, 30, 0);
51      Clock *clockPointer = &meeting;
52
53      (*clockPointer).print24();
54      clockPointer->print24();
55
56      return 0;
57  }
```

```
$ ./clock
09:30:00
09:30:00
```

`this` holds the object's address

`whereAmI()` prints `this`. Called on `meeting`, it prints `meeting`'s address. Called on `lunch`, `lunch`'s.

`this` is a pointer to whichever object the function was called on.

```
$ ./clock
&meeting: 0x7ffd9cacfb40
this: 0x7ffd9cacfb40
&lunch: 0x7ffd9cacfb4c
this: 0x7ffd9cacfb4c
```

```
45  void Clock::whereAmI() {
46      cout << "this: " << this << endl;
47  }
```

```
49  int main() {
50      Clock meeting(9, 30, 0);
51      Clock lunch(12, 15, 0);
52
53      cout << "&meeting: " << &meeting << endl;
54      meeting.whereAmI();
55
56      cout << "&lunch: " << &lunch << endl;
57      lunch.whereAmI();
58
59      return 0;
60  }
```

Now read `this->hours` again

```
meeting.setTime(21, 5, 3);
```



`this` holds `&meeting`

inside `setTime()`, this time:

`this`	means	the address of `meeting`: a pointer
`*this`	means	`meeting` itself
`this->hours`	means	`(*this).hours`, which is `meeting.hours`
`hours`	means	the parameter, 21

Every `->` you've written follows a pointer. `a->b` is always short for `(*a).b`.

Dynamic memory

objects that outlive the function that made them

`new`: make an object on the heap

`new int(137)` sets aside space for an `int` on the **heap**, stores 137 there, and hands back its address.

Objects made with `new` don't have names. The pointer is the only way to reach them.

Every `new` needs a matching `delete` (line 10). More on that shortly.

```
4    int main() {
5        int *myPointer = new int(137);
6
7        cout << "myPointer: " << myPointer << endl;
8        cout << "*myPointer: " << *myPointer << endl;
9
10       delete myPointer;
11
12       return 0;
13   }
```

```
$ ./newInt
myPointer: 0x55c22c35f2b0
*myPointer: 137
```

Returning a heap object from a function

`intPointer` is a local variable: it's gone when `intAllocator()` returns.

The `int` it points to isn't. It lives until `delete` on line 15.

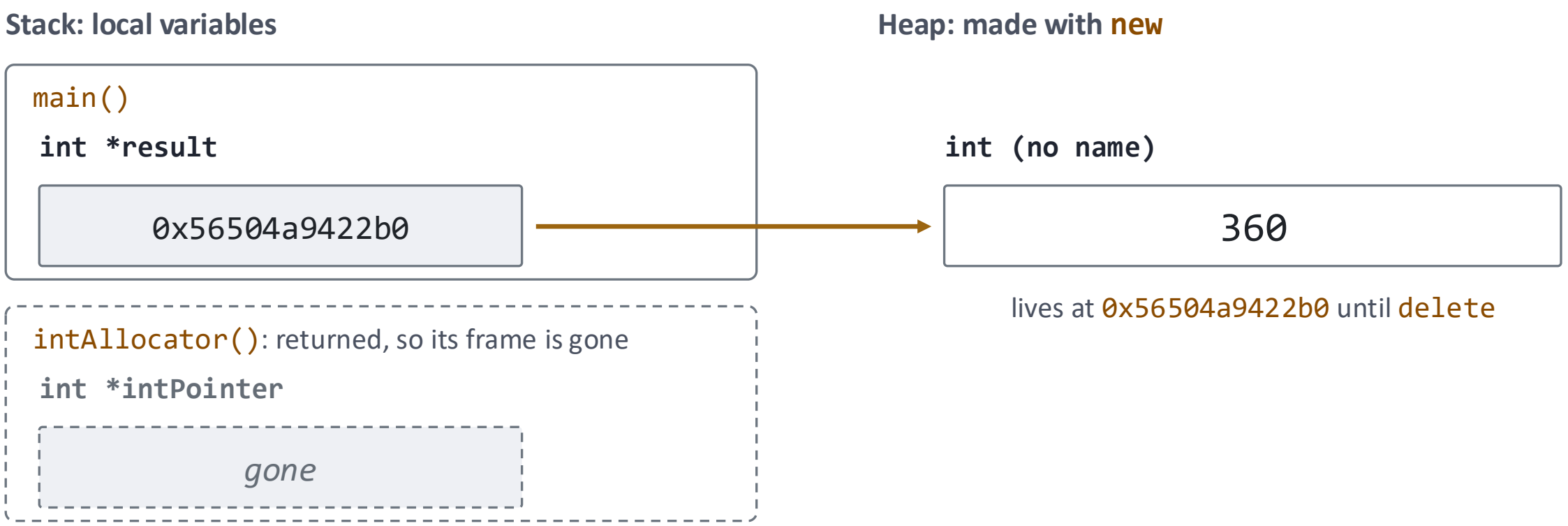
`main()` got a copy of the address, so it can still reach the 360.

```
$ ./intAllocator
intPointer: 0x56504a9422b0
result: 0x56504a9422b0
*result: 360
```

```
1   #include <iostream>
2   using namespace std;
3
4   int *intAllocator(int value) {
5       int *intPointer = new int(value);
6       cout << "intPointer: " << intPointer << endl;
7       return intPointer;
8   }
9
10  int main() {
11      int *result = intAllocator(360);
12      cout << "result: " << result << endl;
13      cout << "*result: " << *result << endl;
14
15      delete result;
16      return 0;
17  }
```

Scope vs. lifetime

Scope is where a *name* can be used. **Lifetime** is how long the *object* exists. For local variables they match. For heap objects they don't: a heap object has no name and no scope, and it lives until `delete`.



Memory leaks

Line 13 overwrites `result`. Nothing holds the 360's address anymore, so nobody can ever `delete` it.

That's a **memory leak**. The output looks fine.

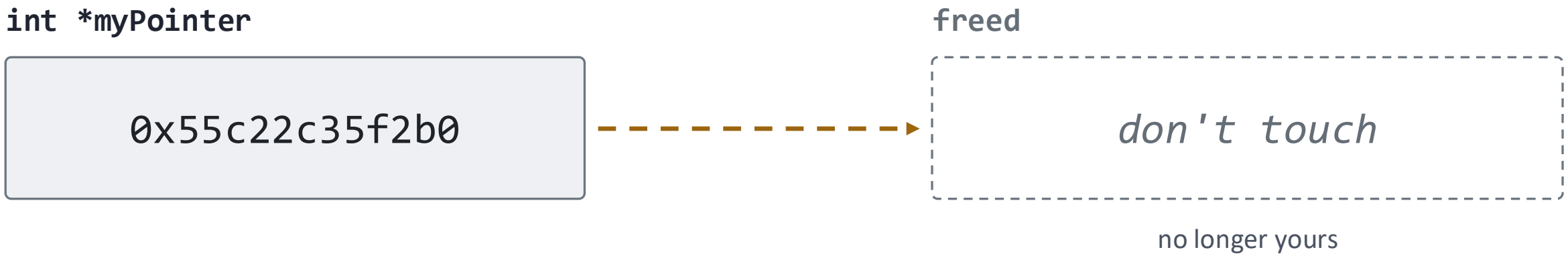
```
10      int *result = intAllocator(360);
11      cout << *result << endl;
12
13      result = intAllocator(137);
14      cout << *result << endl;
15
16      delete result;
17      return 0;
```

`delete` frees the object, not the pointer

`delete myPointer;` gives the `int`'s memory back. `myPointer` itself is untouched: it still holds the old address.

A pointer to memory that's been freed is a **dangling pointer**.

```
delete myPointer;
```



Good habit: `myPointer = nullptr;` right after `delete`. (`delete nullptr;` is safe: it does nothing.)

Predict: a genuinely evil bug

Line 9 deletes the `int`. Line 11 reads it anyway.

What does line 11 print?

```
4    int main() {
5        int *firstPointer = new int(137);
6
7        cout << "before delete: " << *firstPointer << endl;
8
9        delete firstPointer;
10
11       cout << "after delete: " << *firstPointer << endl;
12
13       return 0;
14   }
```

Answer: whatever it likes

Reading freed memory is undefined behavior. Here it printed garbage, different every run.

On another system it might print 137, the old value still sitting there. That's the worst case: it *looks* fine.

```
4    int main() {
5        int *firstPointer = new int(137);
6
7        cout << "before delete: " << *firstPointer << endl;
8
9        delete firstPointer;
10
11       cout << "after delete: " << *firstPointer << endl;
12
13       return 0;
14   }
```

```
$ ./useAfterFree
before delete: 137
after delete: 1541241984
$ ./useAfterFree
before delete: 137
after delete: 1686150095
```

Does setting it to `nullptr` fix it?

Line 9 protects `firstPointer`.

But `secondPointer` still holds the old address. Nulling one pointer can't fix the others, and you can't null out pointers you've forgotten about.

```
4     int main() {
5         int *firstPointer = new int(137);
6         int *secondPointer = firstPointer;
7
8         delete firstPointer;
9         firstPointer = nullptr;
10
11        cout << "secondPointer: " << secondPointer << endl;
12        cout << "*secondPointer: " << *secondPointer << endl;
13
14        return 0;
```

```
$ ./alias
secondPointer: 0x5640012612b0
*secondPointer: 1677726305
```

The rule: one owner for every `new`

Every `new` has one owner

One pointer is responsible for that memory.

The owner `delete`s it exactly once

Zero times is a leak. Twice is undefined behavior.

Nobody uses it after `delete`

Not the owner, and not any other pointer to it.

Other pointers can look, not own

They must not outlive the object they point to.

Setting a pointer to `nullptr` after `delete` is a good habit. Ownership is the actual fix.

Preview: `unique_ptr`

Modern C++ rarely calls `delete` by hand.

A `unique_ptr` *owns* its object and deletes it automatically when it goes out of scope: the ownership rule, enforced for you.

In this class we'll keep writing `new` and `delete` ourselves, so you can see what's happening underneath.

```
$ ./uniq
*myPointer: 137
```

```
1   #include <iostream>
2   #include <memory>
3   using namespace std;
4
5   int main() {
6       unique_ptr<int> myPointer = make_unique<int>(137);
7
8       cout << "*myPointer: " << *myPointer << endl;
9
10      return 0;
11  }
```

Today's vocabulary

pointer

A variable that holds an address.

dereference

Follow a pointer to the thing it points to: `*p`.

`nullptr`

A pointer value that means “points at nothing.”

`->`

`p->x` is short for `(*p).x`.

heap

Where objects made with `new` live, until `delete`.

memory leak

Heap memory nobody can `delete`, because every pointer to it is gone.

dangling pointer

A pointer to memory that's been freed.

undefined behavior

A mistake the C++ standard gives no rules for. Anything can happen.

Next time

pointer arithmetic, and iterators